

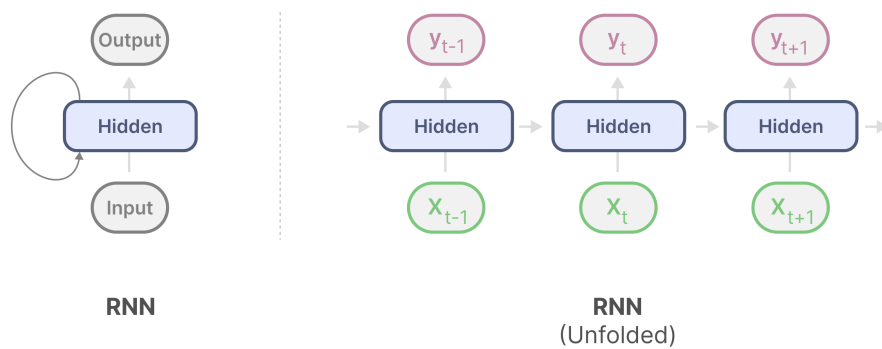
Vision Transformer

1

1

RNN

- Nhắc lại



2

2

Word embedding

- Word embedding: Biểu diễn từ trong không gian vector, mỗi từ được ánh xạ sang một vector số học có chiều cố định
- Vấn đề của RNN/LSTM
 - Word embedding tương ứng với một từ không phụ thuộc vào ngữ cảnh mà từ đó xuất hiện
 - Từ “apple” sẽ cùng vector nhúng dù ngữ cảnh là về trái cây hay về công ty
 - Việc tính toán trạng thái ẩn “Hidden state” diễn ra tuần tự
 - Trạng thái ẩn của một từ phụ thuộc vào trạng thái ẩn của từ từ trước đó
 - Tính trạng thái ẩn của một từ, cần phải chờ tính xong trạng thái ẩn của từ trước
 - Việc tính toán không thể diễn ra đồng thời (song song) cho tất cả các từ mà phải thực hiện tuần tự, từ từ đầu đến cuối của chuỗi văn bản

3

3

Word embedding (2)

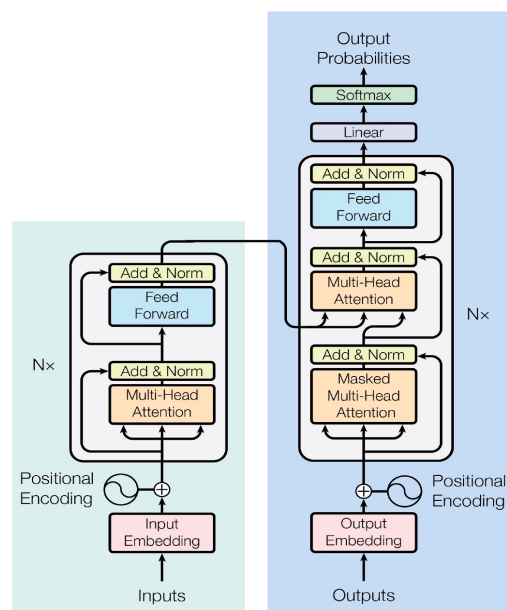
- Transformer
 - Khả năng huấn luyện trước (pre-train) trên một tập dữ liệu lớn
 - Tinh chỉnh (fine-tuning) cho nhiều tác vụ khác nhau
 - Phân loại văn bản, dịch máy
 - Thị giác máy tính nhận diện đối tượng, phân loại hình ảnh, tạo ảnh
 - Tận dụng thông tin lưu trữ ở các bước trung gian/ trạng thái ẩn để xử lý dữ liệu, dựa trên nhu cầu ta có thể
 - Encoder-only: phân loại trích xuất thông tin
 - Decoder-only: sinh văn bản, tự động hoàn thành câu
 - Encoder-decoder: dịch máy
 - Khả năng huấn luyện song song
 - Contextual word embedding: tính toán các biểu diễn từ phụ thuộc ngữ cảnh
 - Multi-head attention: tập trung nhiều phần khác nhau chuỗi đầu vào, mỗi “head” xem xét các khía cạnh khác nhau ngữ cảnh → mô hình khả năng hiểu nhiều mức độ ngữ nghĩa khác nhau
 - Positional encoding: thêm thông tin về vị trí của từ trong chuỗi đầu vào → hiểu rõ ngữ cảnh của từ không chỉ dựa trên từ xung quanh mà còn cả vị trí của chúng trong câu

4

4

Transformer

- Ứng dụng: dịch máy
 - Nguồn: Tiếng anh
 - Đích: Tiếng Pháp
- Trái: Encoder
- Phải: Decoder

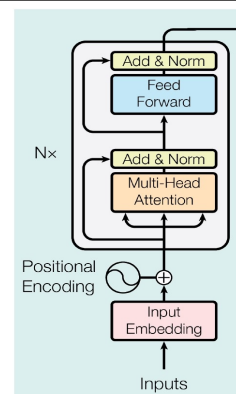


5

5

Encoder block

- Input embedding
 - Lấy các token tương ứng với câu đầu vào trong tiếng Anh
 - Mô hình truyền thống
 - Gán “unknown” token cho từ hiếm gặp
 - Lấy embedding cho các từ phổ biến
 - Transformer
 - Byte pair encoding (tokenization): chia nhỏ các từ thành các phần con
 - Từ hiếm “anand” chia thành ###an và ###and
 - Từ phổ biến “computer” giữ nguyên
 - Không còn từ “unknown”
 - Mỗi token sẽ có embedding tương ứng
 - Tokenization của Transformer đảm bảo những từ hiếm và mới có thể được chia nhỏ thành các thành phần có thể biểu diễn được, thay vì gán cho token “unknown”



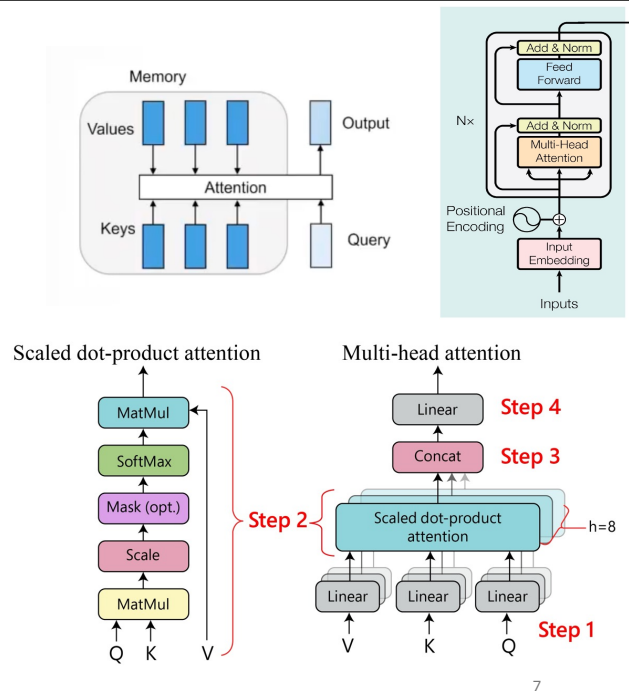
6

6

Encoder (2)

• Self-attention

- Từ các từ được embedding từ pha trước, tạo ra 3 vectors:
 - Query (Q): truy vấn
 - Đại diện từ hiện tại tại mô hình đang xem xét
 - Kiểm tra từ này có liên quan với các từ trong câu không
 - Key (K): khóa
 - Chứa thông tin tất cả các từ khóa trong câu mà mô hình cần đánh giá độ liên quan với từ hiện tại
 - Value (V): giá trị
 - Chứa thông tin mô hình sử dụng tính toán đầu ra.
 - Sau khi tính được mức độ liên quan giữa Query và Key, sử dụng Value để tính giá trị cuối cùng
- Ví dụ: tôi ăn táo → "tôi", "ăn", "táo"
 - Muốn tìm "tôi" liên quan đến "ăn", táo
 - so sánh Query của "tôi" với Key của "ăn" hay "táo"
 - xem được mức độ liên quan giữa các từ,
 - Giả sử "ăn" liên quan "táo" vì ăn táo là hành động có ý nghĩa
 - sau khi xác định quan hệ mạnh giữa "táo" và "ăn",
 - dùng V của từ "ăn" để quyết định đầu ra từ "táo"
 - nhìn từ "ăn" để hiểu rõ hơn "táo"
 - Là đối tượng của hành động ăn

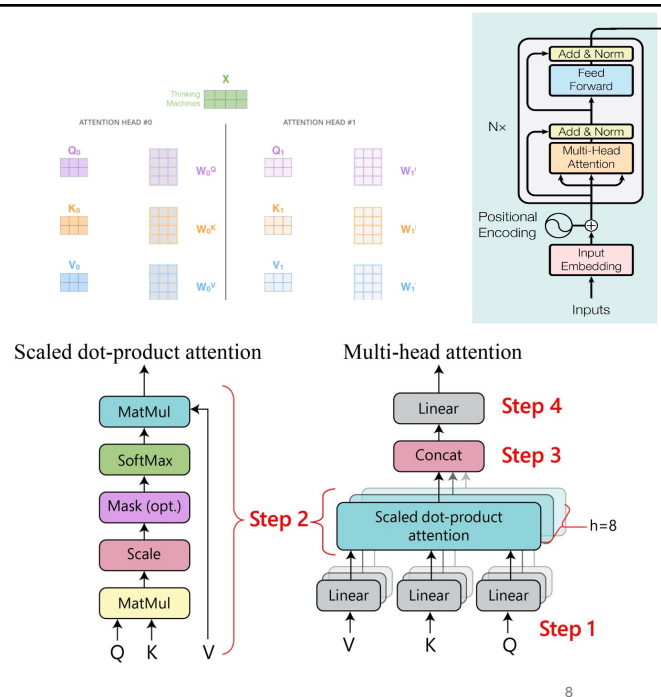


7

Encoder (3)

• Self-attention

- Bước 1: mỗi (Q,V,K) đưa ra 1 góc nhìn khác nhau về semantic
 - Tương tự các filter khác nhau
 - Ví dụ $h=8$, mỗi head xử lý một phần nhỏ Q,V,K
- Bước 2 : Scaled dot-product attention
 - Matmul(Q,K): tích vô hướng, tìm độ liên quan giữa từ đang truy vấn và các từ khác trong câu
 - Scale: Tránh kết quả lớn → chia cho $\sqrt{d_k}$ với d_k là số chiều của K



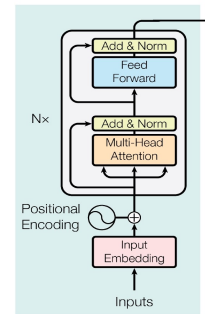
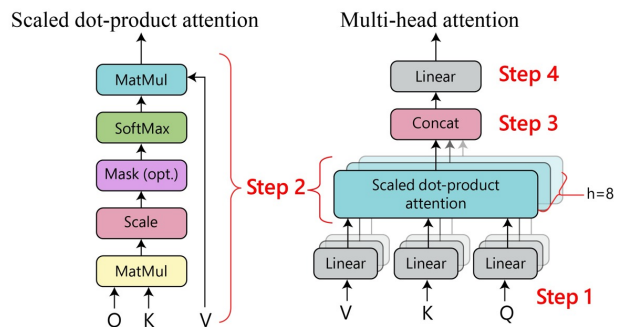
8

Encoder (4)

• Self-attention

- Bước 2 : Scaled dot-product attention
 - Mask (tùy chọn): che bớt một số từ không cần chú ý tới.
 - Ví dụ dịch máy: các từ ở phía sau từ hiện tại chưa cần xử lý tại thời điểm đó.
 - Softmax: chuyển giá trị thành xác suất, cho biết mức độ tập trung vào các từ khác nhau
 - Matmul (đầu ra Softmax, V): tính đầu ra cuối cùng

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



9

9

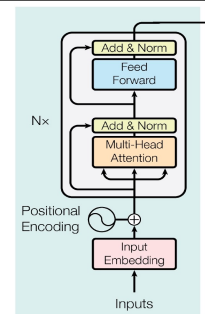
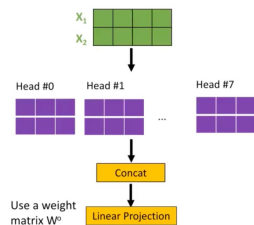
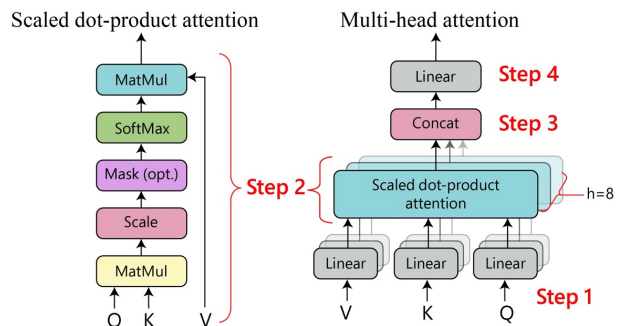
Encoder (5)

• Bước 3 :Concatenate

- Concat đầu ra của các head lại với nhau

• Bước 4: Linear network

- Điều chỉnh kích thước đầu ra, đảm bảo đầu ra giống với đầu vào embedding → tính nhất quán của mô hình



10

10

Encoder (6)

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

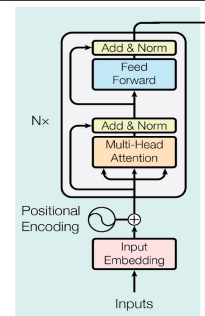
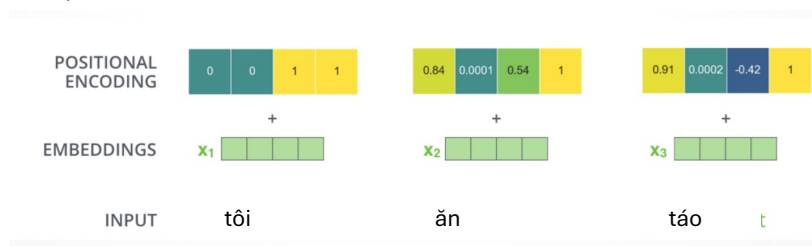
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

• pos is the position and i is the dimension

- Nếu đổi vị trí hai từ ví dụ “ăn táo” với “táo ăn” thì có khác nhau không?

→ Position Encoding (mã hóa vị trí)

- Mỗi vị trí có 1 vector biểu diễn vị trí khác nhau, sử dụng hàm sin, cos

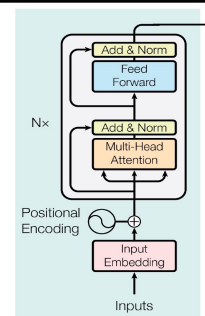


11

11

Encoder (7)

- Add & Norm
 - Đầu ra multi-head attention + đầu vào ban đầu (input embedding và positional encoding)
 - Dùng residual connection (kết nối dư).
 - Bảo toàn thông tin gốc và giúp mô hình ổn định hơn trong quá trình huấn luyện.
- Feed Forward
 - Xử lý các đặc trưng sau khi tính attention
 - Sử dụng ReLU: thêm tính phi tuyến → học các mối quan hệ phức tạp hơn giữa các đặc trưng của từ.
 - Thêm lớp tính toán → tăng khả năng học đặc trưng và khả năng biểu diễn mô hình

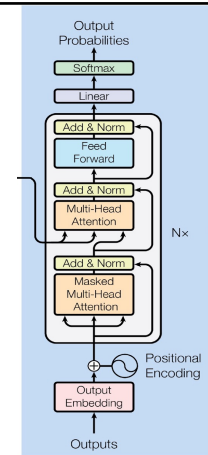


12

12

Decoder

- Masked multi-head attention;
 - Giống multi-head attention của encoder nhưng chỉ chú ý đến từ mà nó sinh ra trước đó chứ không nhìn vào từ tiếp theo, chưa xuất hiện trong quá trình sinh chuỗi đầu ra (che giấu tương lai)
- Cross attention
 - Kết hợp thông tin: decoder lấy thông tin từ encoder để xác định từ tiếp theo
 - K,V từ encoder: mang thông tin đầu vào
 - Q từ decoder: mang thông tin câu đầu ra đang được tạo
 - Điều này giúp mô hình hiểu được cần dự đoán từ gì tiếp theo dựa trên **cả ngữ cảnh của câu gốc**.

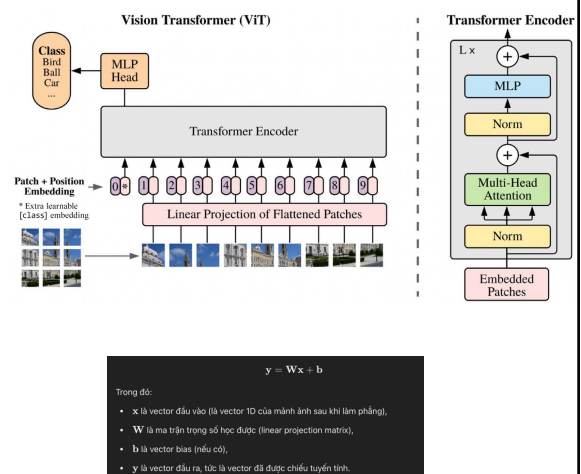


13

13

ViT (1)

- Hình ảnh đầu vào chia thành 9 patches (3*3)
 - Hình gốc kích thước 300*300 thì mỗi mảnh kích thước 100*100
- Làm phẳng và sử dụng phép chiếu tuyến tính (Flatten và Linear projection)
 - Chuyển véc tơ 100*100 thành vector 10,000 phần tử
 - Đưa qua phép chiếu tuyến tính: biến đổi qua một ma trận tuyến tính
 - Điều chỉnh kích thước vector đầu vào thành dạng phù hợp trong xử lý tiếp theo của Transformer
 - Tương tự trích xuất từ nhúng (word embedding) trong NLP: mỗi từ (token) được ánh xạ thành một vector có ý nghĩa và độ lớn nhất định
 - Một patch tương đương với một token → quá trình chiếu tạo vector nhúng cho patch (patch embedding)



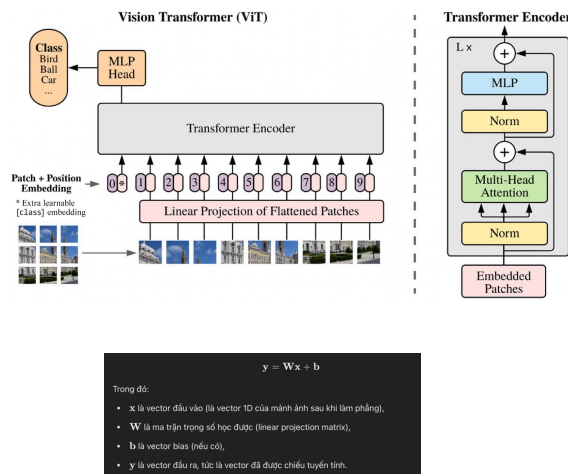
14

14

ViT (2)

• Nhúng vị trí (Position embedding)

- Các patch sau khi được làm phẳng và qua phép chiếu $\rightarrow$ các vector đặc trưng không có thông tin về vị trí gốc của patch
- Sử dụng position embedding
 - Cùng kích thước với vector đặc trưng của patch
 - Được tạo ra dựa vào vị trí từng patch trong ảnh gốc
- Kết hợp position embedding và vector đặc trưng $\rightarrow$ Transformer biết được patch đến từ vị trí nào trong hình ảnh gốc $\rightarrow$ có thông tin về không gian, giúp phân biệt các patch gần nhau có mối liên hệ về ngữ cảnh



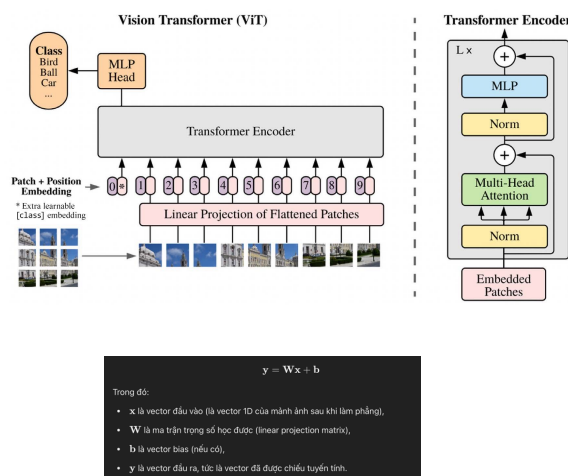
15

15

ViT (3)

• Class embedding (Class token)

- Quan trọng khi phân loại hình ảnh
- Được khởi tạo và thêm vào chuỗi các patch trước khi đưa đến Transformer
- Dùng để tổng hợp thông tin từ các patch khác trong quá trình xử lý
- Sẽ thu thập thông tin từ các patch khác trong bước attention của Transformer
 - Cơ chế attention cho phép class token tương tác với các patch khác và có được thông tin từ các patch này $\rightarrow$ đại diện cho toàn bộ hình ảnh sau Transformer
- Có thể dùng để phân loại hình ảnh

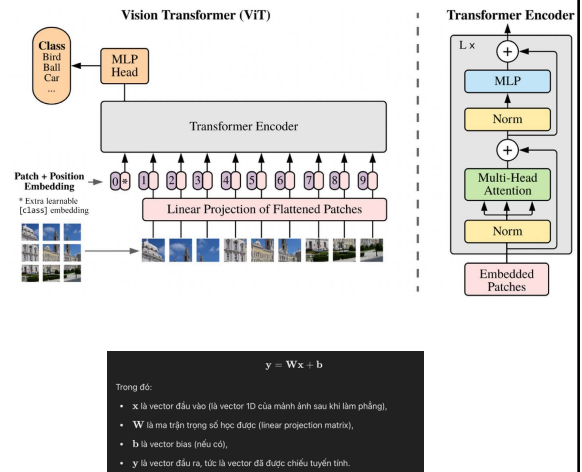


16

16

ViT (4)

- Các vector nhúng được đưa qua Transformer encoder
 - Layer normalization: chuẩn hóa đảm bảo trung bình bằng 0 và phương sai bằng 1 → giảm overfitting
 - Multi-head attention: nhiều phép attention song song cho phép học các khía cạnh khác nhau về các mối quan hệ giữa các patch. Kết quả đầu ra được gộp lại
 - Skip connection: đầu ra attention được kết hợp với đầu vào ban đầu (chuỗi nhúng) → tránh mất mát thông tin
 - MLP: học thêm các mối quan hệ phi tuyến
 - Đầu ra:
 - Class token: Thông tin tổng hợp từ các patch
 - Dùng dự đoán nhãn của ảnh
 - Patch embedding: không chỉ chứa thông tin patch đang xét mà còn thông tin patch khác → học được mối liên hệ giữa các phần tử khác nhau trong ảnh
 - Dùng để làm đặc trưng đầu vào cho các bài toán
 - Nhận dạng đối tượng
 - Chú thích hình ảnh (image captioning): hiểu ngữ cảnh các phần trong ảnh khi kết hợp với mô hình ngôn ngữ

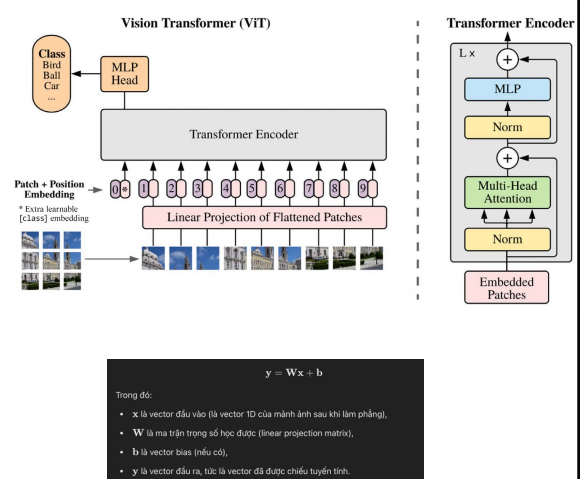


17

17

ViT (5)

- Final head: phụ thuộc vào bài toán.
Ví dụ:
 - MLP head cho bài toán phân loại ảnh
 - Đầu vào là class token đã đi qua Transformer
 - MLP để dự đoán nhãn ảnh
 - Classification head và regression head cho bài toán phát hiện đối tượng
 - Classification head:
 - Đầu vào là patch embedding đã qua Transformer
 - Sử dụng MLP để dự đoán nhãn
 - Bounding box regression head:
 - Đầu vào là patch embedding đã qua transformer
 - Sử dụng MLP để ước tính tọa độ bounding box



18

18

Câu hỏi

1. What are the inputs, steps for calculation, and outputs of self-attention? (Các đầu vào, các bước tính toán, và đầu ra của self-attention là gì?)
2. How is an image transformed into a sequence input in a vision transformer? (Làm thế nào để một hình ảnh chuyển đổi thành chuỗi đầu vào trong ViT)

19

19

Transformer

- Ưu điểm
 - Song song hóa được
 - Có thông tin hai chiều
- Nhược điểm
 - Tốc độ chậm ở inference

20

20

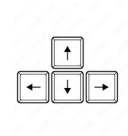
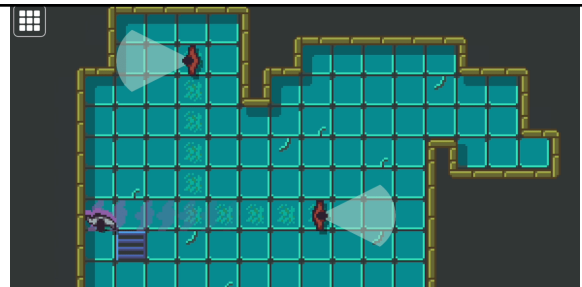
State space model

21

21

Ý tưởng

- Trò chơi làm sao thoát ra cửa
- Các phím điều khiển trái phải lên xuống → sequence input
 - $X_1=R$
 - $X_2=R$
 - $X_3=D$
- Ứng với mỗi vị trí nhân vật trên màn hình: trạng thái → biểu diễn bằng tọa độ (x,y)
 - Trạng thái đầu: X_0 , tính:
 - $X_0=N/A \times s(0)=(1, 3)$
 - $X_1=R \times s(1)=(2, 3)$
 - $X_2=R \times s(2)=(3, 3)$
 - $X_3=D \times s(3)=(3, 2)$
- Làm sao để đi ra ngoài được nếu chỉ có tọa độ?



22

22

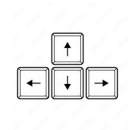
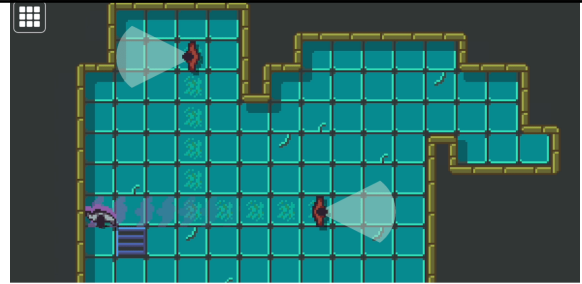
Ý tưởng (2)

- Làm sao để đi ra ngoài được nếu chỉ có tọa độ?

- $X_0 = N/A \times h(0) = (1, 3, 8)$
- $X_1 = R \times h(1) = (2, 3, 7)$
- $X_2 = R \times h(2) = (3, 3, 6)$
- $X_3 = D \times h(3) = (3, 2, 5)$

- Thêm giá trị mới: khoảng cách từ nhân vật đến cửa

- Ở vị trí đầu đi 9 bước mới đến cửa
- Đi sang phải còn 8 bước mới đến cửa
- Cửa ra nằm ở đâu?
 - Còn 9 bước nữa đến cửa ra
 - Xu hướng đi xuống về bên phải
- $h(4) = ??? (4, 2, 4)$
- $h(\text{final}) = (7, 1, 0)$



23

23

State space model (SSM)

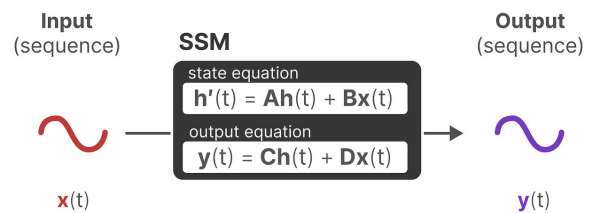
- Trên thực tế
 - Tọa độ không còn rời rạc, mà liên tục
 - Hướng đi không còn 4 hướng mà là 1 biến liên tục
- SSM: mô hình mô tả các biểu diễn trạng thái (state representations) và đưa ra dự đoán về trạng thái tiếp theo dựa trên chuỗi đầu vào nhất định, Ứng với mỗi input sẽ có 1 state



24

24

SSM (2)

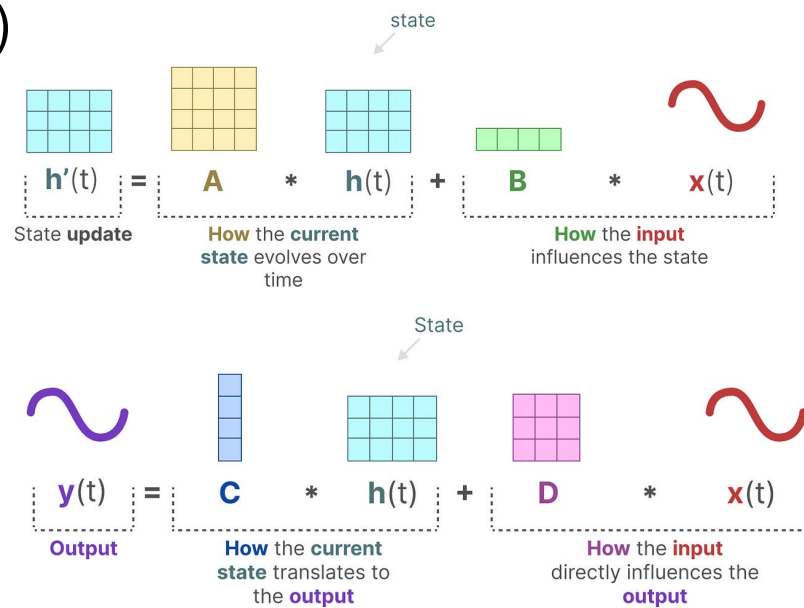


- Giả định: hệ thống có thể được dự đoán từ trạng thái tại thời điểm t bằng hai phương trình
- Phương trình trạng thái
 - A: ma trận chuyển trạng thái: mối quan hệ trạng thái hiện tại và trạng thái tiếp theo
 - h' : đạo hàm trạng thái tại thời điểm t (cho biết hệ thống thay đổi như thế nào tại thời điểm t)
 - B: ma trận đầu vào, mô tả ảnh hưởng của $x(t)$ tới $h(t)$
- Phương trình đầu ra
 - Y: đầu ra
 - C: ma trận đầu ra, mô tả quan hệ giữa trạng thái $h(t)$ và đầu ra y
 - D: ma trận tác động của đầu vào trực tiếp tới đầu ra
- Mục tiêu: tìm $h(t)$, từ đó đoán được $y(t)$ dựa trên $x(t)$

25

25

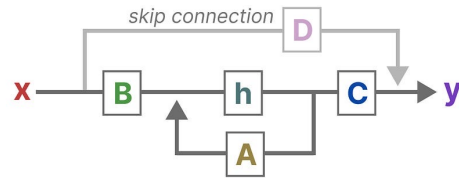
SSM (3)



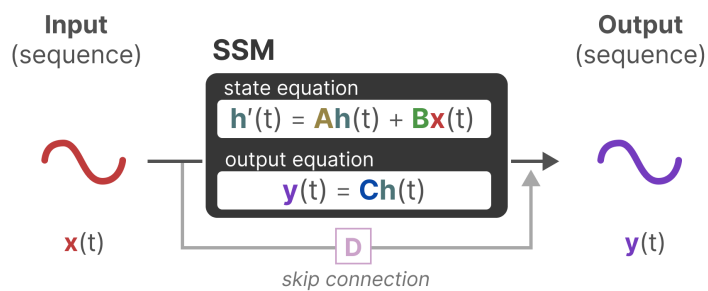
26

26

SSM (4)



- Cách biểu diễn đơn giản hơn, bỏ qua Skip connection D

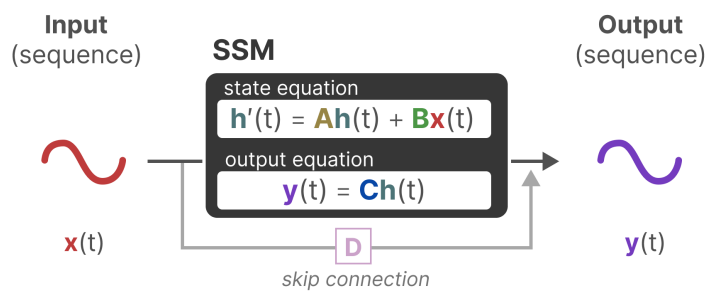


27

27

SSM (5)

- Công thức tính không có activation function, đơn giản hơn
- Vật lý chứng minh: phương trình này luôn có lời giải dù không dùng activation function khi tất cả các biến nằm trong miền liên tục

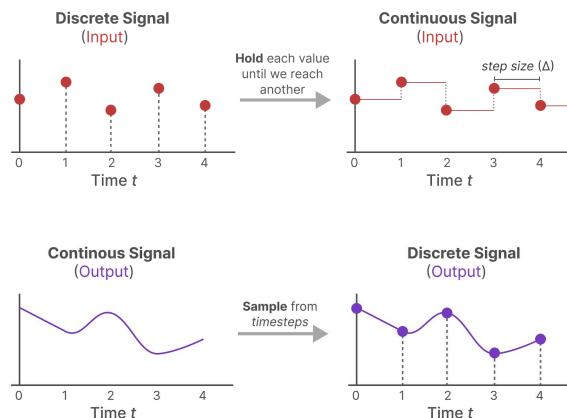


28

28

SSM (6)

- Dữ liệu chúng ta là rời rạc, ví dụ chuỗi ký tự → cần xử lý



- Zero-order hold technique
 - Tín hiệu đầu vào rời rạc: giữ nguyên giá trị cho đến khi nhận được tín hiệu rời rạc mới → tạo ra giá trị liên tục
 - Thời gian giữ giá trị delta: độ phân giải của đầu vào
- Có tín hiệu liên tục cho đầu vào, sẽ có đầu ra liên tục, ta lấy mẫu theo thời gian để có giá trị rời rạc

29

29

SSM (7)

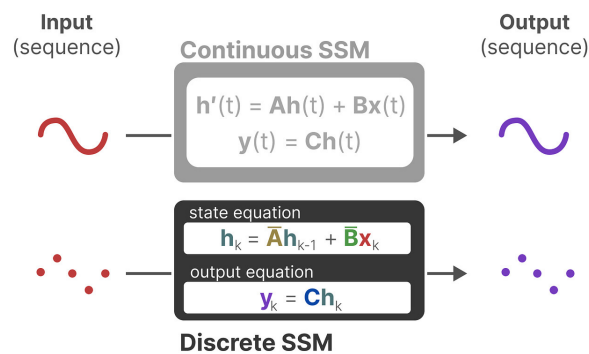
- Theo toán học, tính được

Discretized matrix **A**

$$\bar{\mathbf{A}} = \exp(\Delta \mathbf{A})$$

Discretized matrix **B**

$$\bar{\mathbf{B}} = (\Delta \mathbf{A})^{-1} (\exp(\Delta \mathbf{A}) - \mathbf{I}) \cdot \Delta \mathbf{B}$$



30

30

SSM (8)

- SSM giải quyết được cả vấn đề của RNN và Transformer
- Transformer
 - Ưu
 - Có thể xử lý thông tin dày đặc trong một cửa sổ ngữ cảnh (context windows):
Xử lý và truyền tải thông tin một cách hiệu quả trong phạm vi (context window, nơi các từ hoặc yếu tố có liên quan gần được xem xét cùng lúc)
 - **Song song** hóa trong huấn luyện
 - Tăng tốc độ huấn luyện: tính toán không cần phải phụ thuộc vào các bước tuần tự (như RNN truyền thống)
 - Nhược
 - Dự đoán chậm (**slow inference**)
 - Khi dự đoán: độ phức tạp cao, đặc biệt khi độ dài của chuỗi dữ liệu tăng → Dự đoán chậm dần, với độ phức tạp tính toán tỷ lệ với bình phương độ dài chuỗi
 - Không thể mô hình hóa thông tin ngoài cửa sổ ngữ cảnh:
 - Chỉ có thể xử lý dữ liệu bên trong cửa sổ ngữ cảnh đã định → Không thể tận dụng thông tin quan trọng nằm **ngoài phạm vi context windows** để đưa ra dự đoán

31

31

SSM (9)

- SSM giải quyết được cả vấn đề của RNN và Transformer
- RNN
 - Ưu
 - Dự đoán nhanh (fast inference)
 - Dự đoán trong RNN có tính **tuyến tính (linear)**, nghĩa là thời gian tính toán tăng theo cách tuyến tính với độ dài chuỗi dữ liệu
 - Có thể xử lý ngữ cảnh lớn (**large context length**) trong lý thuyết
 - RNN có thể xử lý thông tin trong các chuỗi dài với cửa sổ ngữ cảnh lớn, vì nó sử dụng cơ chế tuần tự, mỗi bước trong chuỗi dựa vào thông tin từ các bước trước đó
 - Nhược
 - Dễ quên thông tin theo thời gian
 - xu hướng **quên** dần các thông tin quan trọng khi chuỗi dữ liệu quá dài, đặc biệt là với các phiên bản RNN cơ bản.
 - Không thể huấn luyện song song
 - RNN phải xử lý từng bước dữ liệu theo thứ tự tuần tự. Điều này gây khó khăn cho việc song song hóa quá trình huấn luyện, làm cho huấn luyện RNN chậm hơn so với các mô hình có khả năng huấn luyện song song như Transformers.

32

32

SSM (10)

- Tận dụng cả Transformer và RNN

	Training	Inference
Transformers	Fast! (parallelizable)	Slow... (scales quadratically with sequence length)
RNNs	Slow... (not parallelizable)	Fast! (scales linearly with sequence length)

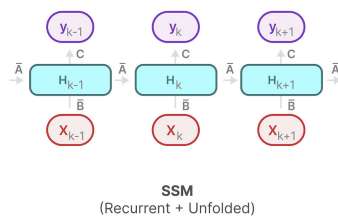
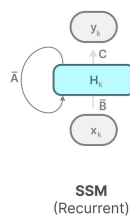
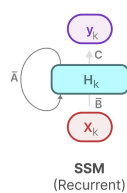
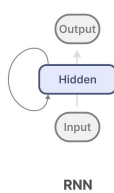
33

33

Hai góc nhìn của SSM

- Tận dụng: train một đường/ inference một đường khác

- Train theo cơ chế Transformers
- Inference theo cơ chế RNN
 - Không có activation function
 - Nhân ma trận
 → nhanh



34

34

Hai góc nhìn của SSM (2)

- Làm sao huấn luyện song song?
- CNN

$$h_0 = Bx_0$$

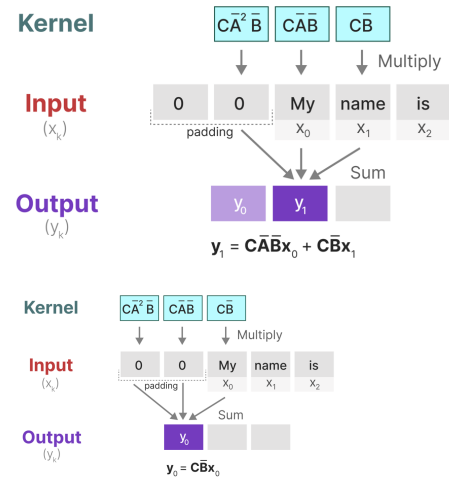
$$y_0 = Ch_0 = C\bar{B}x_0$$

$$h_1 = \bar{A}h_0 + \bar{B}x_1 = \bar{A}\bar{B}x_0 + \bar{B}x_1$$

$$y_1 = Ch_1 = C(\bar{A}\bar{B}x_0 + \bar{B}x_1) = C\bar{A}\bar{B}x_0 + C\bar{B}x_1$$

$$h_2 = \bar{A}h_1 + \bar{B}x_2 = \bar{A}(\bar{A}\bar{B}x_0 + \bar{B}x_1) + \bar{B}x_2 = \bar{A}^2\bar{B}x_0 + \bar{A}\bar{B}x_1 + \bar{B}x_2$$

$$y_2 = Ch_2 = C(\bar{A}^2\bar{B}x_0 + \bar{A}\bar{B}x_1 + \bar{B}x_2) = C\bar{A}^2\bar{B}x_0 + C\bar{A}\bar{B}x_1 + C\bar{B}x_2$$



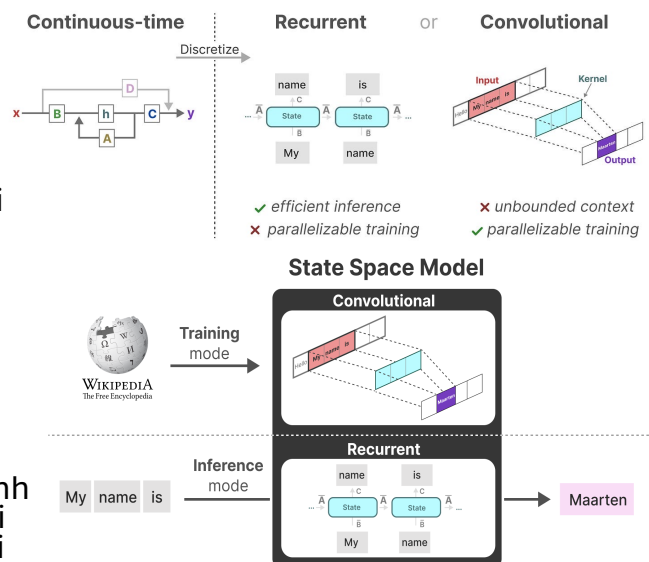
35

35

Hai góc nhìn của SSM (3)

- Mô hình tuyến tính bất biến theo thời gian (linear time invariant): hệ thống phản hồi với đầu vào không đổi theo thời gian
- Tất cả các tham số cố định theo thời gian: A, B, C được giữ nguyên bất kể trạng thái hay ở bước thời gian nào

→ Mô hình biểu diễn tĩnh (static representation) mà không nhận thức được nội dung (content aware): mô hình không điều chỉnh tham số dựa trên nội dung hoặc ngữ cảnh đang xử lý tại thời điểm đó



36

36

Quan trọng của ma trận A

Produces hidden state

$$h_k = \bar{A}h_{k-1} + \bar{B}x_k$$

$$y_k = Ch_k$$

- Việc tạo ra ma trận A đúng cách giúp hệ thống nhớ không chỉ một vài token trước đó mà còn toàn bộ các token đã xử lý đến thời điểm hiện tại
 - Nếu ma trận A không tạo ra tốt: trong biểu diễn hồi quy, chỉ nhìn lại trạng thái trước đó (một bước thời gian trước) và không có khả năng giữ lại thông tin trong quá trình dài
- Làm sao A giúp duy trì bộ nhớ lớn (context size) và giữ được thông tin từ các bước trước?

37

37

Quan trọng của ma trận A (2)

Produces hidden state

$$h_k = \bar{A}h_{k-1} + \bar{B}x_k$$

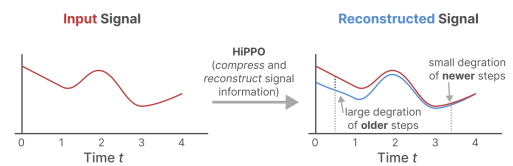
$$y_k = Ch_k$$

- Sử dụng HiPPO (High-order Polynomial Projection Operators)
 - Phương pháp giúp nén và tái tạo lại các tín hiệu đầu vào mà hệ thống đã thấy cho đến thời điểm hiện tại.
 - Nó cố gắng giữ lại và nén thông tin đầu vào thành một **vector các hệ số (vector of coefficients)**
 - **Tái tạo tín hiệu (reconstruct)**: Sau khi nén, HiPPO sử dụng các hệ số đó để tái tạo lại tín hiệu, cố gắng khôi phục tín hiệu sao cho gần với bản gốc nhất có thể.

38

38

Quan trọng của ma trận A (3)



- **Bên phải:** tín hiệu đã được tái tạo từ quá trình nén.
 - Màu đỏ: tín hiệu đầu vào ban đầu
 - Màu xanh là tín hiệu đã được tái tạo lại
 - **Suy giảm tín hiệu cũ (large degradation of older steps):**
 - Có sự suy giảm rõ rệt đối với các phần tín hiệu cũ hơn (ở bên trái biểu đồ)
 - các giá trị của tín hiệu trong quá khứ bị mất đi phần nào độ chính xác sau khi tái tạo
 - **Suy giảm tín hiệu mới nhỏ (small degradation of newer steps):**
 - Các bước mới hơn (bên phải đường màu xanh) được tái tạo lại khá chính xác với mức suy giảm rất nhỏ
- phù hợp với việc mô hình hồi quy cần nhớ thông tin quan trọng gần nhất để đưa ra dự đoán, nhưng không nhất thiết phải nhớ toàn bộ quá trình với độ chính xác cao

39

39

Quan trọng của ma trận A (4)

HiPPO Matrix

1	0	0	0
1	2	0	0
1	3	3	0
1	3	5	4

k

n

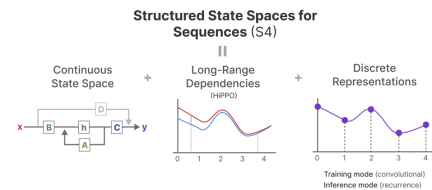
- Ma trận HiPPO:
 - Hàng bắt đầu là 1
 - Các số tăng dần theo đường chéo
 - Giá trị 0 xuất hiện phần trên bên phải của ma trận
 - Ma trận tam giác dưới
 - Hàng phía dưới đại diện cho thông tin gần hơn
 - Ý nghĩa
 - Tiết kiệm bộ nhớ: Không cần lưu chi tiết mọi thông tin cũ
 - Tập trung vào hiện tại: Thông tin gần đây được lưu đầy đủ hơn
 - Hiệu quả trong dự đoán: Kết hợp được cả thông tin chi tiết gần đây và xu hướng tổng quan từ quá khứ

40

40

Cấu trúc State space cho chuỗi

- Dùng xử lý chuỗi dữ liệu dài
- Kết hợp
 - Không gian trạng thái liên tục: hệ thống liên tục thay đổi trạng thái theo thời gian khi có đầu vào mới (xử lý dữ liệu liên tục)
 - Khả năng xử lý chuỗi dài hạn HiPPO: nén và quản lý các chuỗi dữ liệu dài không mất đi tính chính xác các thông tin quan trọng (nhớ bắt được mối quan hệ phức tạp)
 - Các biểu diễn rời rạc: huấn luyện nhanh chóng trong chế độ song song, nhưng vẫn có khả năng xử lý tốt các chuỗi dài hạn trong chế độ suy luận tuần tự (linh hoạt trong quá trình huấn luyện và suy luận)



41

41

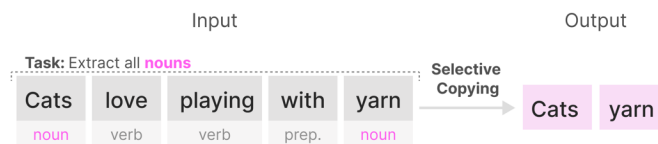
Mamba

42

42

Ý tưởng

- S4 kém trong một vài tác vụ
 - Selective copyting (sao chép chọn lọc): ví dụ trích xuất danh từ
 - Linear time invariant: xử lý mọi token trong chuỗi dữ liệu theo cùng một cách, không phân biệt ngữ cảnh hay loại từ.
 - A,B,C của mô hình S4 là không đổi
 - Mô hình xử lý mọi token như nhau dù là danh từ, động từ hay từ khác

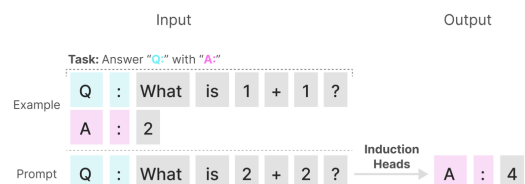


43

43

Ý tưởng (2)

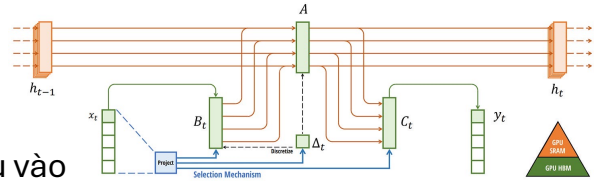
- S4 kém trong một vài tác vụ
 - Induction head: tái tạo các mẫu (patterns) xuất hiện trong dữ liệu đầu vào
 - Trả lời câu hỏi Q bằng A và mô hình dựa vào câu hỏi phía trước để trả lời câu hỏi hiện tại
 - Hỏi 1+1 → trả lời 2: mô hình được cung cấp câu hỏi và câu trả lời, cần ghi nhớ để trả lời
 - Đặt câu hỏi mới: 2+2: mô hình phải tự trả lời theo mẫu câu hỏi trước đáp án là 4
 - Vấn đề SSM
 - Tuyến tính bất biến theo thời gian (Time invariant): sử dụng chung một quy tắc xử lý, không có ngữ cảnh, không ghi nhớ được thông tin quan trọng ví dụ 1+1=2



44

44

Selective SSM

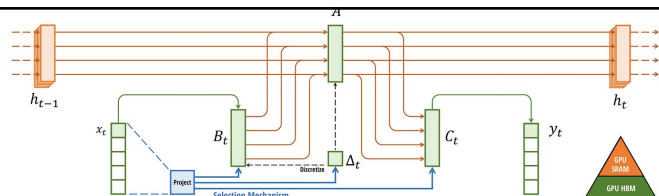


- Làm cho B,C và delta phụ thuộc đầu vào
 - Step size delta:
 - Tham số rời rạc hóa
 - Delta nhỏ: tập trung vào ngưỡng cảnh nhiều hơn, step size lớn tập trung vào từ đầu vào, ít quan tâm ngưỡng cảnh
 - Ma trận B:
 - Cách đầu vào tác động đến hidden state
 - Đối với mỗi token đầu vào, ma trận B thay đổi để xử lý ngưỡng cảnh chính xác hơn thay vì giữ nguyên như mô hình tuyến tính bất biến theo thời gian
 - Ma trận C:
 - Quyết định cách hidden state hiện tại ảnh hưởng đến đầu ra
 - Phụ thuộc vào đầu vào giúp xử lý mỗi token theo ngưỡng của riêng từng token
- Sử dụng cơ chế gating
 - Quyết định thông tin nào cần giữ lại và thông tin nào nên được loại bỏ

45

45

Selective SSM (2)



- GPU Memory Hierarchy (Phân cấp bộ nhớ GPU):
 - CPU/SWAP (Màu cam): Tầng trên
 - Bộ nhớ CPU và không gian swap
 - Dung lượng lớn nhưng tốc độ chậm hơn
 - GPU HBM (Màu xanh lá): Tầng dưới
 - High Bandwidth Memory của GPU
 - Tốc độ nhanh hơn nhưng dung lượng nhỏ hơn
 - Được sử dụng cho tính toán trực tiếp

47

47

Vision mamba

- Ảnh đầu vào:

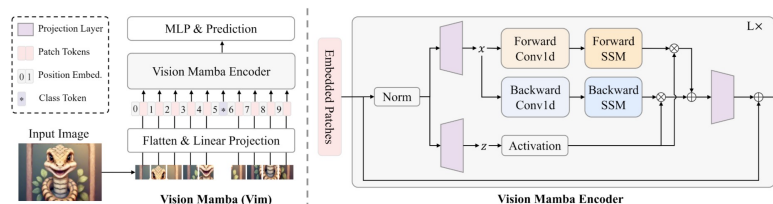
- Hình ảnh đầu vào được chia thành các **patch tokens** (các khối nhỏ) bằng cách chia nhỏ hình ảnh thành các phần khác nhau
- Mỗi phần được biểu diễn dưới dạng **patch token** và qua quá trình **Flatten & Linear Projection** (biến đổi tuyến tính)

- Position embedding:

- Được thêm vào để giữ thông tin vị trí của mỗi patch trong hình ảnh

- Class token:

- Token đặc biệt được thêm vào để biểu diễn toàn bộ hình ảnh cho mục đích phân loại hoặc dự đoán.



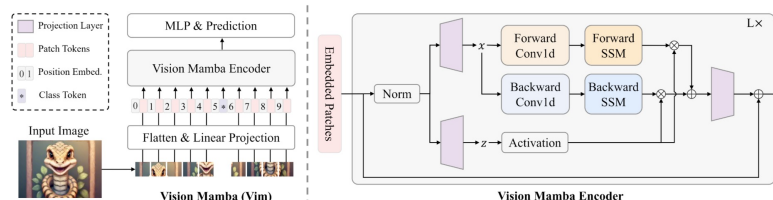
48

48

Vision mamba (2)

- Vision mamba encoder

- Phần chính xử lý chuỗi các patch token đã qua position embedding
- Áp dụng L khối, với mỗi khối, sử dụng thông tin cả hai chiều:
 - Forward conv1D và backward conv1D: nắm bắt các đặc trưng ngắn, xử lý dữ liệu ngắn
 - **Forward SSM** và **Backward SSM**: thông tin ngữ cảnh dài
 - Sự kết hợp giữa **Conv1d** và **SSM** giúp mô hình trích xuất thông tin theo hai chiều: không gian và thời gian, cải thiện khả năng xử lý chuỗi dài
 - **Norm** và **Activation**: đảm bảo các thông tin được xử lý ổn định và hiệu quả.



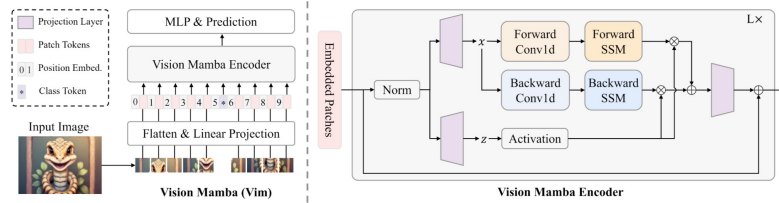
49

49

Vision mamba (3)

• MLP & Prediction

- Sau khi dữ liệu được xử lý qua các lớp của **Vision Mamba Encoder**
→ được đưa vào một **MLP**
- **MLP** sẽ kết hợp các đặc trưng trích xuất từ nhiều lớp để tạo ra dự đoán cuối cùng cho các nhiệm vụ như phân loại hoặc nhận dạng đối tượng



50